# Artificial Intelligence

## Assignment 4 Report

### Topic: Greedy Scheduling and A* Search for Optimal Assignment Planning

---

**Group A**

---


**Submitted By**

| | |
|---|---|
| **Karan Veer Gautam** | 2301MC10 |
| **Prakhar Palod** | 2301MC17 |
| **Prashant Nigam** | 2301MC18 |

Department of Mathematics
Indian Institue Of Technology Patna


March 20, 2026

# Contents


**1 Introduction** **3**

**2 Problem Formulation** **3**

**3 Question 1: Greedy Schedule Generation** **4**

3.1 Objective . . . . . 4
3.2 Greedy Strategies Used . . . . . 4
3.2.1 Greedy by Food Cost . . . . . 4
3.2.2 Greedy by Dependency Depth . . . . . 4
3.3 Methodology . . . . . 5
3.4 Correctness of the Greedy Schedule . . . . . 5
3.5 Results for Question 1 . . . . . 5
3.6 Sample Schedule Summary . . . . . 6
3.7 Discussion . . . . . 6
3.8 Important Graphs for Question 1 . . . . . 6
3.8.1 Assignment Dependency Graph . . . . . 6
3.8.2 Total Days and Total Cost Comparison . . . . . 6
3.8.3 Day-wise Number of Assignments Scheduled . . . . . 7
3.8.4 Schedule Timeline . . . . . 8
3.9 Inference from Question 1 . . . . . 8

**4 Question 2: A* Search for Optimal Schedule** **9**

4.1 Objective . . . . . 9
4.2 Why A* Search? . . . . . 9
4.3 State-Space Representation . . . . . 9
4.4 Transition Function . . . . . 9
4.5 Cost Function . . . . . 10
4.6 Heuristic Function . . . . . 10
4.7 Methodology . . . . . 10
4.8 Results for Question 2 . . . . . 10
4.9 Comparison with Greedy . . . . . 11
4.10 Important Graphs for Question 2 . . . . . 11
4.10.1 Greedy vs A* Total Cost Comparison . . . . . 11
4.10.2 Greedy vs A* Total Days Comparison . . . . . 11
4.10.3 Day-wise Cost Comparison . . . . . 12
4.11 Inference from Question 2 . . . . . 12

**5 Complexity Analysis** **13**
5.1 Greedy Approach . . . . . 13
5.2 A* Approach . . . . . 13

**6 Implementation Details** **13**

**7 Conclusion** **13**

**8 Future Work** **14**

# 1 Introduction

The assignment focuses on solving a constrained scheduling problem where a group of students must complete a set of assignments. Each assignment:

- requires specific prerequisite outcomes,
- produces an output outcome,
- requires a specific food item, and
- must respect a maximum group size constraint per day.

The problem can be viewed as a scheduling problem on a directed acyclic graph (DAG), where:

- nodes represent assignments,
- edges represent dependency relationships,
- only available assignments can be scheduled,
- at most $g$ assignments can be done in one day.

Two tasks were solved:

1. **Question 1:** Generate a valid schedule using greedy heuristics.
2. **Question 2:** Use A* search to compute an optimal schedule.

# 2 Problem Formulation

The given input consists of:

- cost of each food item,
- group size $g$,
- initial input resources,
- final target outputs,
- list of assignments with prerequisites, output, and food type.

Each assignment is of the form:

$$A_i = (\text{prerequisite}_1, \text{prerequisite}_2, \text{output}, \text{food item})$$

An assignment can be scheduled only if both of its prerequisites are already available. After completion, its output becomes available for future assignments.

The objectives are:

- generate valid schedules,
- compute total food cost,
- compute total number of days,
- compare heuristic and optimal approaches.

# 3 Question 1: Greedy Schedule Generation

## 3.1 Objective

The goal of Question 1 is to generate valid schedules using greedy heuristics while satisfying:

- dependency constraints,
- daily group-size limit,
- food requirement per assignment.

## 3.2 Greedy Strategies Used

Two greedy strategies were implemented.

### 3.2.1 Greedy by Food Cost

At each day, all currently available assignments are identified. From these, assignments having the lowest food cost are selected first, up to the group-size limit.

**Tie-breaking used:**

- lower food cost,
- smaller assignment index.

### 3.2.2 Greedy by Dependency Depth

In this strategy, priority is given to assignments lying on deeper dependency chains. The depth of an assignment is defined as the length of the longest path from that assignment to a terminal assignment.

This strategy attempts to unlock more future work earlier and is therefore structurally more informed than a purely cost-based heuristic.

### 3.3 Methodology

The greedy algorithm follows these steps:

1. Parse the input instance.
2. Store assignment details and food costs.
3. Maintain a set of currently available resources.
4. Identify all assignments whose prerequisites are satisfied.
5. Rank available assignments according to the chosen greedy heuristic.
6. Select at most $g$ assignments for the day.
7. Update completed assignments and newly available outputs.
8. Repeat until all assignments are completed.

### 3.4 Correctness of the Greedy Schedule

A schedule is valid if an assignment is performed only after all its prerequisites are available. The implemented greedy procedure ensures correctness by selecting assignments only from the set of currently available assignments. Therefore, every generated schedule is feasible.

### 3.5 Results for Question 1

For the given sample instance:

- both greedy strategies produced valid schedules,
- both respected all prerequisite constraints,
- both satisfied the group-size constraint,
- both completed the full assignment set.

Since all food items in the provided sample have equal cost, both strategies resulted in the same total food cost. However, the assignment ordering differed.

### 3.6 Sample Schedule Summary

Table 1: Summary of Greedy Strategies

| Strategy | Total Days | Total Cost |
|---|---|---|
| Greedy by Food Cost | 6 | 11 |
| Greedy by Dependency Depth | 6 | 11 |

### 3.7 Discussion

Although both strategies produced the same total cost on the sample input, this is mainly because all food costs were equal. The dependency-depth strategy is still more informative because it prioritizes assignments that unlock longer downstream chains. In cases with unequal food costs, the behaviour of the two strategies can differ significantly.

### 3.8 Important Graphs for Question 1

#### 3.8.1 Assignment Dependency Graph

The dependency graph visually represents prerequisite relationships among assignments and helps explain why some assignments must be scheduled earlier than others.

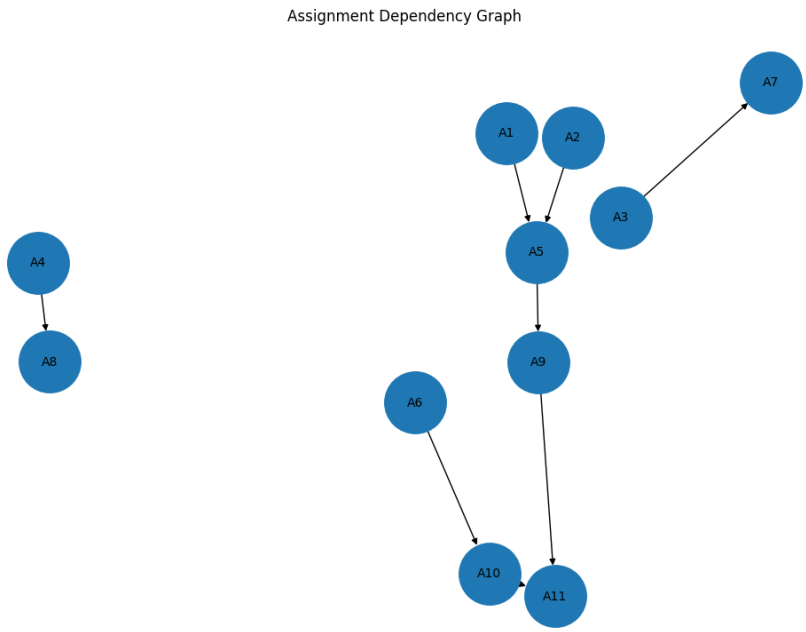


Figure 1: Assignment dependency graph for the given problem instance

#### 3.8.2 Total Days and Total Cost Comparison

This graph compares the final performance of both greedy strategies in terms of total days and total food cost.

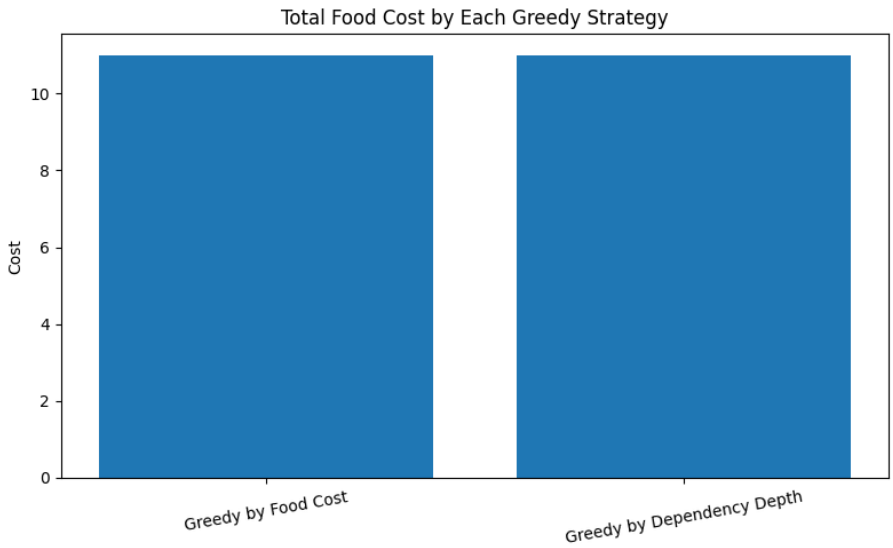


Figure 2: Comparison of total cost for greedy strategies

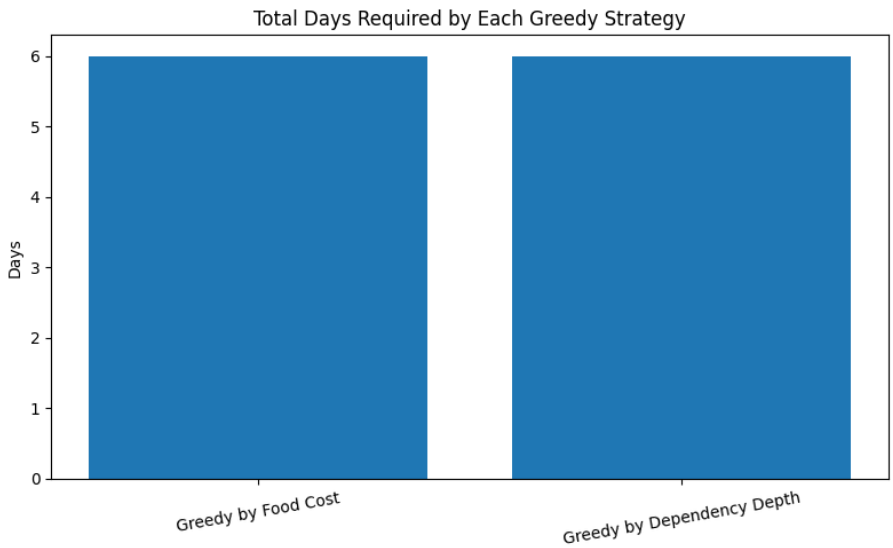


Figure 3: Comparison of total days for greedy strategies

### 3.8.3 Day-wise Number of Assignments Scheduled

This plot shows how many assignments were completed each day under both strategies.

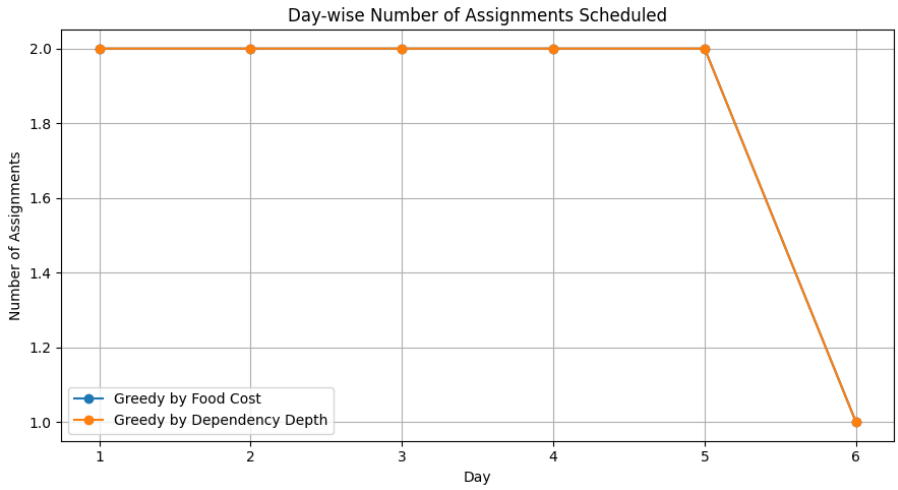


Figure 4: Day-wise number of assignments scheduled

### 3.8.4 Schedule Timeline

The schedule timeline provides a clear visual representation of the order in which assignments were scheduled.

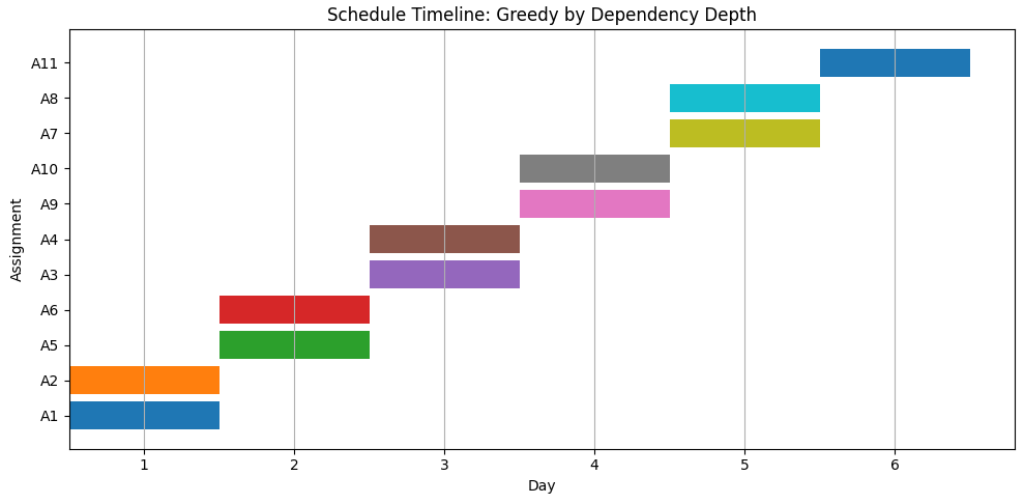


Figure 5: Timeline of assignment execution under greedy strategies

## 3.9 Inference from Question 1

The following conclusions were drawn:

- Greedy methods are simple and efficient for generating valid schedules.
- Food-cost greedy is useful when food prices differ significantly.
- Dependency-depth greedy better captures the structure of the dependency graph.
- For the sample input, both strategies achieved identical aggregate results due to equal food costs.

# 4 Question 2: A* Search for Optimal Schedule

## 4.1 Objective

The objective of Question 2 is to find an optimal schedule using A* search. Unlike greedy methods, which make local decisions, A* explores the state space systematically to guarantee an optimal solution with respect to total food cost.

## 4.2 Why A* Search?

A* is an informed search algorithm that uses both:

- the actual cost incurred so far, and
- a heuristic estimate of the remaining cost.

It evaluates states using:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ is the actual path cost from the start state to state $n$,
- $h(n)$ is the estimated cost from state $n$ to the goal,
- $f(n)$ is the estimated total solution cost passing through state $n$.

## 4.3 State-Space Representation

A state in the A* search contains:

- set of completed assignments,
- set of currently available resources,
- schedule built so far,
- total cost accumulated so far.

The goal state is reached when all assignments have been completed.

## 4.4 Transition Function

From a current state:

1. find all currently available assignments,
2. generate all valid subsets of size 1 to $g$,
3. each subset represents one possible next-day action,
4. apply the chosen action to produce the next state.

### 4.5 Cost Function

The actual cost used in A* is:

$$g(n) = \text{total food cost accumulated so far}$$

This directly corresponds to the optimization objective.

### 4.6 Heuristic Function

The heuristic used is:

$$h(n) = (\text{number of remaining assignments}) \times (\text{minimum food cost})$$

This is admissible because each remaining assignment must incur at least one food item cost, and the minimum food cost is a lower bound on future cost. Since the heuristic never overestimates the actual remaining cost, A* is guaranteed to find an optimal solution.

### 4.7 Methodology

The A* algorithm follows these steps:

1. Initialize the start state with no completed assignments and only initial resources available.
2. Insert the start state into a priority queue.
3. Expand the state with the smallest $f(n)$.
4. If all assignments are completed, return the schedule.
5. Otherwise, generate successor states using all valid next-day assignment combinations.
6. Compute $g(n)$, $h(n)$, and $f(n)$ for each successor.
7. Continue until the goal state is reached.

### 4.8 Results for Question 2

The A* approach successfully generated an optimal valid schedule. For the sample input:

- total food cost was optimal,
- all dependency constraints were satisfied,
- the schedule was feasible and complete.

Since the sample input assigns equal food cost to all assignments, the optimal total cost matches the number of assignments completed. Therefore, A* and greedy may appear similar on this sample instance, but A* remains the correct globally optimal method in general.

### 4.9 Comparison with Greedy

Table 2: Comparison of Greedy and A*

| Method | Total Days | Total Cost |
|---|---|---|
| Best Greedy Strategy | 6 | 11 |
| A* Search | 6 | 11 |

### 4.10 Important Graphs for Question 2

#### 4.10.1 Greedy vs A* Total Cost Comparison

This graph compares the overall total cost produced by greedy scheduling and A* search.

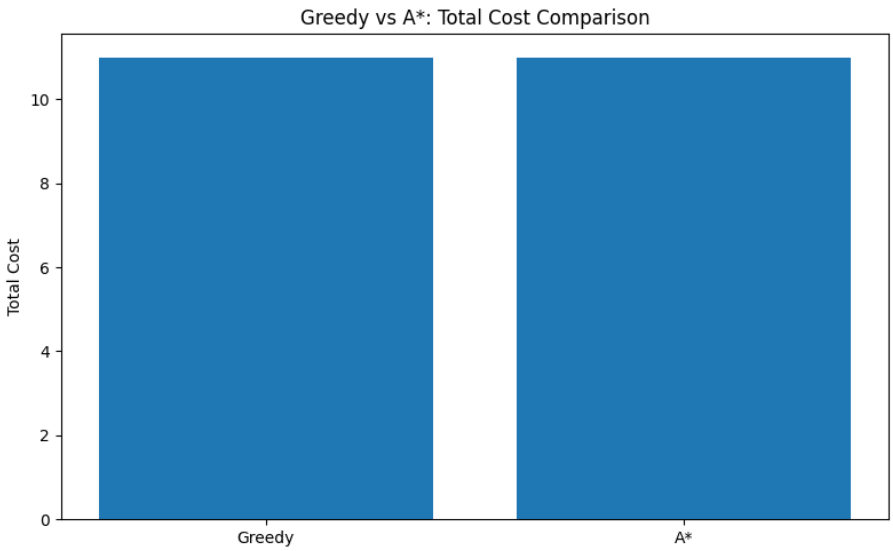


Figure 6: Total cost comparison between Greedy and A*

#### 4.10.2 Greedy vs A* Total Days Comparison

This graph compares the total number of days required by both methods.

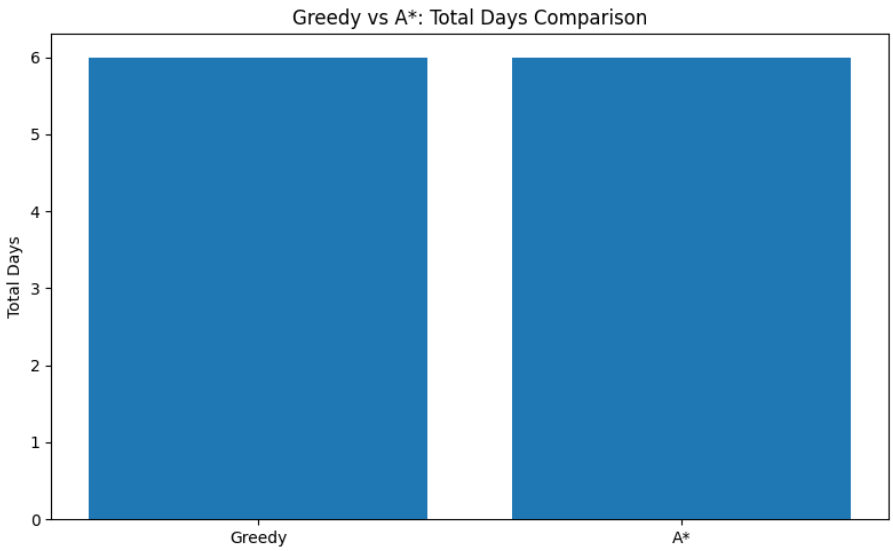


Figure 7: Total days comparison between Greedy and A*

### 4.10.3 Day-wise Cost Comparison

This plot compares daily cost progression of greedy and A* schedules.

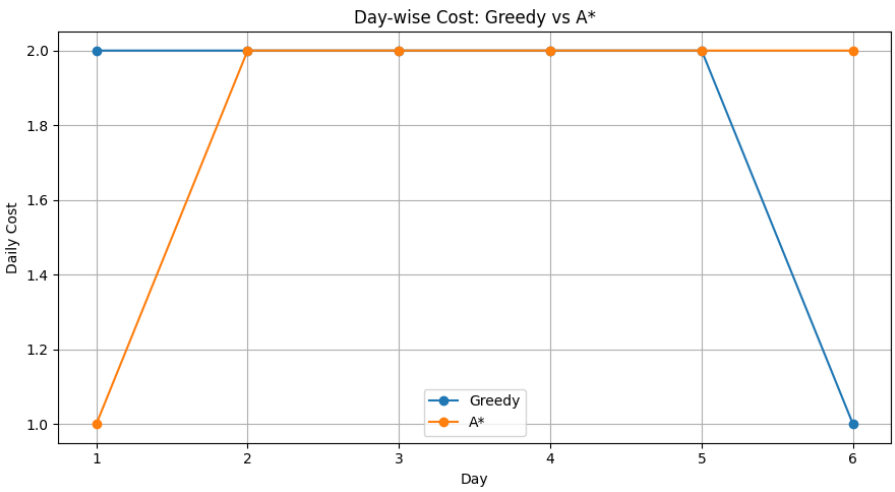


Figure 8: Day-wise food cost comparison between Greedy and A*

## 4.11 Inference from Question 2

The following observations were made:

- A* provides a principled way to search for the globally optimal schedule.
- The admissible heuristic ensures optimality.
- For the sample input, A* and greedy give similar total cost because all food costs are equal.
- In general, A* is more reliable than greedy for optimization problems.

# 5 Complexity Analysis

## 5.1 Greedy Approach

For each day, the algorithm scans all assignments to determine which are available and then sorts the available assignments according to the selected heuristic. If $n$ is the number of assignments, the worst-case complexity is approximately:

$$O(n^2 \log n)$$

which is acceptable for small to medium problem sizes.

## 5.2 A* Approach

The A* algorithm explores a state space consisting of partial schedules. Its worst-case complexity is exponential because multiple valid successor states may be generated at each step. However, the heuristic reduces unnecessary exploration by guiding the search toward promising states.

# 6 Implementation Details

The entire assignment was implemented in Python and executed in Google Colab. The following components were developed:

- input parser,
- dependency graph builder,
- greedy schedulers,
- A* search implementation,
- helper functions for daily menu and cost computation,
- visualization functions for report plots.

The implementation was modular, allowing both Question 1 and Question 2 to reuse the same parsing and state representation logic.

# 7 Conclusion

This assignment demonstrated two important AI problem-solving paradigms:

- **Greedy search**, which is fast and simple but makes local decisions,

- **A* search**, which is more computationally expensive but guarantees optimality when used with an admissible heuristic.

For Question 1, two greedy strategies were implemented successfully and both generated valid schedules. For Question 2, A* search was used to find the optimal schedule. The results showed that while greedy methods can perform well on simple instances, A* is more robust for optimization under constraints.

This assignment also highlighted the importance of graph-based reasoning, heuristic design, and structured result analysis through tables and visualizations.

## 8 Future Work

The following improvements can be considered:

- testing the algorithms on larger and more varied inputs,
- analysing behaviour when food costs differ significantly,
- designing stronger heuristics for A*,
- exploring dynamic programming or branch-and-bound based methods,
- optimizing memory usage for large state spaces.

## References


1. Assignment 4 problem statement provided in the course.
2. Class notes on greedy algorithms and informed search.
3. Python documentation for data structures and algorithmic implementation.